OpenDesign author-time daemon + MCP wiring — reviewed rootless-container setup recipe

Revision: 1 **Last modified:** 2026-07-09T00:00:00Z **Status:** REVIEWED — ready for operator/next-session adoption (containerized, localhost-only) **Authority:** Helix OTA §11.4.161 (rootless containers) · §11.4.76 (containers submodule) · §11.4.173 (containerized/distributed build) · §11.4.162 (OpenDesign is the design system) · §11.4.99 (latest-source verification) **Scope of this document:** a vetted, ready-to-execute recipe. It is a REVIEWED RECIPE, not an execution log — nothing here was built or run. Building the image is a heavy, security-sensitive step and stays NEEDS-REVIEW / operator-gated. **Ground truth:** `docs/research/opendesign_integration_20260709/INTEGRATION_GROUND_TRUTH.md` (W1 verdict: author-time half = `od` daemon + Next.js UI + MCP server; NEEDS-REVIEW because the daemon is a network service).

---

0. TL;DR verdict

SAFE TO ADOPT — containerized, rootless, localhost-only, with the hardening in §5 applied.

The upstream `deploy/docker-compose.yml` is already hardened to a degree most projects never reach (read-only rootfs, `no-new-privileges`, non-root uid 1001, `pids_limit`, `mem_limit`, tmpfs `/tmp`, and a **`127.0.0.1`-only** port publish). It maps almost 1:1 onto a rootless-podman run. The daemon also ships a real, DNS-aware SSRF guard and a bind-token safety floor (verified in source, §4).

The residual risk is **not** the network surface — it is that the daemon's job is to **spawn coding-agent runtimes (Claude Code / Codex / opencode) as child processes** to do design work. That is arbitrary-code-execution *by design*, contained by the container sandbox. Two concrete things must be controlled by the operator: (a) never adopt the `docker-compose.linux.yml` host-networking + host-credential-mount

override unless a nested agent runtime is actually required, and (b) if it is required, treat the mounted `~/.claude` credentials + `*_API_KEY` env as the blast radius and scope them per §5.

---

1. Source-of-truth files (all under the read-only clone …`/scratchpad/open-design/`)

Concern	File	What it establishes
Base compose (hardened)	`deploy/docker-compose.yml`	`127.0.0.1:${PORT}:7456`, `read_only: true`, `security_opt: [no-new-privileges:true]`, `mem_limit`, `pids_limit: 256`, tmpfs `/tmp`, named volume `open_design_data:/app/.od`, healthcheck. `OD_BIND_HOST: 0.0.0.0` **inside** the container (the `127.0.0.1` host publish is what confines it).
Image build	`deploy/Dockerfile`	Multi-stage `node:24-alpine`; native builds (`better-sqlite3`, `python3/make/g++`); runtime stage drops to non-root user `open-design` (uid/gid 1001), `tini` init, `poppler-utils bash git`. Entry: `node apps/daemon/dist/cli.js --no-open`.
Env template	`deploy/.env.example`	`OD_API_TOKEN` (bearer for non-loopback), `OPEN_DESIGN_DISABLE_API_AUTH`, `OPEN_DESIGN_ALLOWED_ORIGINS`, `OPEN_DESIGN_MEM_LIMIT`, `OD_CODEX_SANDBOX`. Explicit note: “Keep Compose bound to localhost; use an authenticated reverse proxy, SSH tunnel, or VPN before exposing remotely.”
Linux CLI-mount	`deploy/docker-compose.linux.yml`	`network_mode: host`, `cpus: '1.5'`, forces `OD_BIND_HOST:`

Concern	File	What it establishes
override (**high-risk**)		`127.0.0.1`, mounts host agent CLIs + **`~/.claude` credentials (ro)** + `ANTHROPIC_API_KEY`/ `OPENAI_API_KEY`/ `DEEPSEEK_API_KEY`.
Deploy guidance	`deploy/README.md`	“Do not publish the daemon directly on a public or shared LAN interface. The API is unauthenticated for non-browser clients.” Image “intentionally does not bundle Claude/Codex/Gemini CLI binaries.”
Bind-token safety floor	`apps/daemon/src/server.ts` (≈L1923–1956)	Refuses to bind a non-loopback host unless `OD_API_TOKEN` is set or `OD_DISABLE_API_AUTH=1`. Loopback (`127.0.0.1`/`::1`/ `localhost`) always allowed tokenless.
Bearer-token auth	`apps/daemon/src/api-token-auth.ts` + `server.ts` bearer middleware	`Authorization: Bearer <OD_API_TOKEN>` enforced on `/api/*` when a token is set; loopback origins exempt; health/readiness/version stay open.
Origin / same-origin guard	`apps/daemon/src/http/origin-guard.ts`, `apps/daemon/src/origin-validation.ts`	Browser-origin + Host-header validation; `OD_ALLOWED_ORIGINS` allowlist; `sec-fetch-site` handling; clipper extension carve-out narrowly scoped.
SSRF guard (real)	`apps/daemon/src/origin-validation.ts` (`isPrivateIpv4`, `isLoopbackOrPrivateLanHost`, `configuredAllowedInternalHosts`) + `apps/daemon/src/connectionTest.ts` (`assertExternalAssetUrl`, DNS-aware re-resolution)	Default-deny to RFC1918 / loopback / link-local (`169.254`); **re-resolves DNS names** and re-checks every resolved address (anti-DNS-rebinding); documented loopback carve-out for local LLMs; operator opt-in `OD_ALLOWED_INTERNAL_HOSTS`

Concern	File	What it establishes
		(CIDR rejected loudly, never silently trusted).
Reasoning egress policy	`apps/daemon/src/reasoning-egress.ts`	Per-run allowlist for outbound LLM provider egress (`enabled`/`disabled`/`allowlist`), base-URL + model allowlisting.
MCP → agent installer	`apps/daemon/src/mcp-agent-install.ts`	For `claude` it shells `claude mcp add --scope user <name> -e <ENV> -- <command> <args>` (uses Claude Code's own merge rules, does not hand-edit config). JSON-merge strategy for cursor/cline/etc.; print-only for unverified formats.
MCP launch spec	`apps/daemon/src/mcp-install-info.ts` (`buildMcpInstallPayload`)	Wires `command=<node execPath>`, `args=[<cli.js>, mcp, --daemon-url, http://127.0.0.1:<port>]`, `env={OD_DATA_DIR,…}`. Daemon URL is **loopback only**.
MCP stdio server (tool surface)	`apps/daemon/src/mcp.ts`	Stateless stdio bridge; "holds no state and never touches the filesystem; every tool resolves to a `fetch()` against `OD_DAEMON_URL`." Tools: `list_projects`, `get_active_context`, `get_project`, `create_project`, `delete_project`, `list_files`, `get_file`, `search_files`, `get_artifact`, `create_artifact`, `write_file`, `delete_file`, `list_skills`, `list_plugins`, `start_run`, `get_run`, `cancel_run`. 30-min idle self-exit.

2. Rootless-podman recipe (§11.4.161 / §11.4.173)

2.0 Preconditions (host, one-time — no root)

- Rootless podman + `podman-compose` (or `podman compose` provider) present and on PATH.
- cgroups v2 with **systemd user delegation** so `mem_limit`/`pids_limit` are honored rootless. Verify:

```
cat /sys/fs/cgroup/user.slice/user-$(id -u).slice/user@$(id
        -u).service/cgroup.controllers   # must list: cpu
        memory pids
podman info --format '{{.Host.CgroupsVersion}}
        {{.Host.CgroupControllers}}'
```

  If `memory`/`pids` are absent, `mem_limit`/`pids_limit` become no-ops (podman warns) — do NOT silently accept an unbounded container; enable delegation first (`loginctl enable-linger $USER`, drop-in `Delegate=memory pids cpu`). This composes with §12.6 (60% mem ceiling) — the container MUST stay bounded.

- **Do NOT run any of this under sudo/root** (§11.4.161).

2.1 Preferred path: through the `vasic-digital/containers` submodule (§11.4.76)

The containers submodule is the sanctioned orchestration layer and already speaks rootless `podman-compose` first-class:

- `containers/pkg/compose` — `NewOrchestrator("podman-compose", …)` (see `containers/pkg/compose/podman_compose_test.go`, `orchestrator_test.go`). It knows the podman-compose quirks (e.g. it must NOT emit the unsupported `--wait` flag) and parses `podman-compose ps` JSON.
- `containers/pkg/runtime` auto-detects Docker → Podman; force Podman rootless.
- `containers/pkg/volume`, `pkg/network`, `pkg/health` cover the volume/health lifecycle.

Adoption pattern (do NOT hand-roll bare `podman` calls per §11.4.161): register OpenDesign's compose file as a Helix-managed compose group with runtime pinned to `podman-compose`, pointing at a **copy** of `deploy/docker-compose.yml` placed under a Helix-owned deploy dir (never

mutate the read-only clone). The orchestrator's `Up/Down/Status` then drives the lifecycle. (The submodule is a Go module; the exact wiring call lives in Helix deploy code, out of scope for this research doc — this recipe fixes the compose + env inputs it consumes.)

2.2 Underlying mechanism / fallback: bare `podman-compose`

Documented as the mechanism the submodule invokes and the escape hatch if the submodule path is unavailable. Pull the **pre-built** image (avoids the heavy native build entirely — §2.4):

```
# 1. Copy upstream compose + env into a Helix-owned deploy dir
        (never edit the read-only clone)
mkdir -p ~/helix-opendesign && cd ~/helix-opendesign
cp <clone>/deploy/docker-compose.yml ./docker-compose.yml
cp <clone>/deploy/.env.example ./.env

# 2. Generate a bearer token even for localhost (defense in depth;
        harmless on loopback)
printf 'OD_API_TOKEN=%s\n' "$(openssl rand -hex 32)" >> ./.env

# 3. Pin the image by DIGEST (not the mutable :latest) in .env,
        e.g.
#    OPEN_DESIGN_IMAGE=ghcr.io/nexu-io/od@sha256:<digest>
#    (resolve the digest with: podman manifest inspect ghcr.io/
        nexu-io/od:<version>)

# 4. Bring up, rootless, localhost-only
podman-compose --env-file ./.env -f ./docker-compose.yml up -d

# 5. Verify it is bound to loopback ONLY and healthy
podman-compose -f ./docker-compose.yml ps
ss -ltnp | grep 7456          # MUST show 127.0.0.1:7456, never
        0.0.0.0:7456
curl -s http://127.0.0.1:7456/api/health
```

Rootless-specific notes for this exact compose: - **Port** `127.0.0.1:${OPEN_DESIGN_PORT:-7456}:7456` — publishes on host loopback only. Rootless podman honors the `127.0.0.1` host-IP prefix. **Verify with `ss -ltnp` every launch** (§4 anti-bluff — do not assume). - **Named volume** `open_design_data` lands under `~/.local/share/containers/storage/volumes/` and podman auto-chowns it to the container user (uid 1001 → your subuid range). No `:U`/manual chown normally needed for a named volume; only add `:U` if a permission error appears. - **`read_only: true` + tmpfs `/tmp`** — works rootless; the daemon writes only to the `/app/.od` named volume and `/tmp`. - **`security_opt: no-new-`

`privileges:true` — works rootless; keep it. - **`OD_BIND_HOST: 0.0.0.0`** inside the container is fine ONLY because the host publish is `127.0.0.1`. Do NOT "fix" it to a LAN IP.

2.3 Wait-for-health before declaring up (§11.4.5 evidence, not assumption)

The compose healthcheck runs `node -e "fetch('http://127.0.0.1:7456/api/health')…"`. Gate any downstream step on `podman healthcheck run open-design` / `podman-compose ps` reporting healthy, plus a real `curl /api/health` 200 — never on "up -d returned 0".

2.4 Building the image (NEEDS-REVIEW — do NOT run on bare host autonomously)

`deploy/Dockerfile` performs native compilation (`better-sqlite3`, `sharp` transitively, `python3/make/g++`) and a full pnpm workspace build — heavy and long. Per §11.4.173 this MUST run inside a build container on the designated remote build host, artifacts brought back — never a bare-host `podman build`. **Prefer pulling the published `ghcr.io/nexu-io/od@sha256:<digest>` image** (§2.2 step 3) and skip building entirely. If a self-built image is genuinely required, route it through the containers-submodule `crossbuild/distribution` path and treat it as its own operator-gated work item.

---

3. Wiring the MCP server into Claude Code

Key correctness fact: `od mcp install claude` bakes the `node` execPath + `cli.js` path of **wherever it runs**. Run it *inside* the container and it wires the container's throwaway `~/.config`, not your host Claude Code. So for a **containerized daemon + host Claude Code**, do NOT run the in-container installer. Wire the host agent to a stdio bridge that reaches the loopback daemon. The bridge (`apps/daemon/src/mcp.ts`) is stateless and only proxies to `OD_DAEMON_URL`.

3.1 Recommended (fully containerized bridge, no host node/od needed)

Point Claude Code's MCP at a `podman exec -i` into the running container. The bridge runs inside the container's netns, so its daemon URL `http://127.0.0.1:7456` is the daemon itself:

```
claude mcp add --scope user open-design -- \
  podman exec -i open-design node apps/daemon/dist/cli.js mcp --
        daemon-url http://127.0.0.1:7456
```

- `-i` attaches stdio (MCP stdio transport). Claude Code spawns/respawns this on demand.
- No host Node.js, no host `od` CLI, no extra open port — the MCP path never leaves loopback + a local `podman exec`.
- UNCONFIRMED: interaction of the bridge's 30-min idle self-exit (`MCP_STDIO_IDLE_EXIT_MS`) with Claude Code's stdio-server respawn was not runtime-tested here; MCP clients normally respawn stdio servers, so this is expected-benign but should be confirmed on first real use.

3.2 Alternative (host-side od CLI bridge)

If you install a host `od` CLI (Node) separately, you may instead run the installer on the host pointed at the containerized daemon:

```
od mcp install claude --daemon-url http://127.0.0.1:7456
# → shells: claude mcp add --scope user open-design -e
        OD_DATA_DIR=<…> -- <node> <od cli.js> mcp --daemon-url
        http://127.0.0.1:7456
```

Only adopt this if you already want a host `od` CLI; otherwise §3.1 keeps everything in the container.

3.3 What the MCP surface grants the outer agent (review this before wiring)

Via the 17 tools in `mcp.ts`, a Claude Code session gains, **scoped to Open Design projects under the daemon data dir** (not arbitrary host paths): - read: `list_projects`/`get_project`/`list_files`/`get_file`/`search_files`/`get_artifact`/`list_skills`/`list_plugins`/`get_active_context`; - mutate: `create_project`/`delete_project`/`create_artifact`/`write_file`/`delete_file`; - **commission a nested agent**:

`start_run` — "Open Design spawns its own agent to do the work". This is the MCP path into the daemon's runtime-spawning surface (§4). `delete_project`/`delete_file` are destructive within the project store.

There is **no raw-shell tool** on the MCP surface. The code-execution capability is the daemon's `start_run` runtime spawning, contained by the container.

---

4. Security review

4.1 Network exposure — LOW as configured, contingent on localhost bind

- Listens on **one** TCP port, default `7456`, serving both `/api` and the static Next.js export. Base compose publishes it on `127.0.0.1` only.
- **Bind-token safety floor is real** (`server.ts` ≈L1940): the daemon *refuses to start* on a non-loopback bind without `OD_API_TOKEN` (or explicit `OD_DISABLE_API_AUTH=1`). So a misconfig to `0.0.0.0` fails closed unless a token is set — good.
- Honest gap the vendor states plainly: **the API is unauthenticated for non-browser clients on loopback** (bearer + origin checks exempt loopback so the local UI works). Anything with loopback access to `7456` (any local user / another container sharing the netns) can call `/api/*`. Mitigation: keep it a single-user host and localhost-only; do not run with `network_mode: host` on a shared box.

4.2 Authentication — adequate for localhost, must be tightened for any exposure

- Bearer `OD_API_TOKEN` enforced on `/api/*` when set; loopback exempt. Health/readiness/version open (fine for probes).
- `OD_DISABLE_API_AUTH=1` is a real bypass — only for a trusted reverse proxy that authenticates upstream. Treat setting it as a §11.4.122-class decision.

4.3 SSRF gate — REAL and defense-in-depth (verified in source)

- Literal-IP block: `isPrivateIpv4` covers `10/8`, `172.16–31`, `192.168/16`, `169.254/16`; `isLoopbackOrPrivateLanHost` adds loopback + `0.0.0.0`/`::`.

- **DNS-aware**: `connectionTest.ts` re-resolves the hostname and re-runs the block-list against **every resolved address** — this defeats a public DNS name pointing at internal infra / DNS-rebinding (the comment calls this out explicitly).
- `assertExternalAssetUrl` guards the attacker-controllable asset-download path and is **not** widened by the operator internal-host allowlist.
- Operator opt-in `OD_ALLOWED_INTERNAL_HOSTS` exists for a deliberately-reachable internal LLM; **CIDR entries are rejected loudly**, malformed entries dropped with a warning — it cannot silently widen.
- Reasoning-provider egress is separately gated by an allowlist policy (`reasoning-egress.ts`).
- Verdict: the SSRF posture is stronger than typical. Keep the default (empty `OD_ALLOWED_INTERNAL_HOSTS`).

4.4 The actual blast radius — the daemon spawns coding-agent runtimes

- The daemon's design function is to run agent runtimes (Claude Code / Codex / opencode) as child processes to generate designs (`src/runtimes/`, `start_run`). That is arbitrary code execution **by design**.
- In the plain container this is confined: non-root uid 1001, read-only rootfs, `no-new-privileges`, `pids_limit`, `mem_limit`, no host mounts, loopback-only.
- **`OD_CODEX_SANDBOX=danger-full-access`** (documented escape hatch when Codex's own workspace-write sandbox can't init in an unprivileged container) **removes Codex's inner sandbox** — broader filesystem access *inside the container*. Do not set it unless required, and only on a trusted single-user host.
- **`docker-compose.linux.yml` is the high-risk mode**: `network_mode: host` (drops the bridge's port isolation; compensated only by `OD_BIND_HOST=127.0.0.1`), and it mounts host **`~/.claude` credentials (ro)** + injects `ANTHROPIC_API_KEY`/`OPENAI_API_KEY`/`DEEPSEEK_API_KEY`. A nested agent runtime then executes with those live credentials mounted. If you never need in-container agent CLIs, **do not use this override at all** — the plain `docker-compose.yml` has none of these mounts.

4.5 MCP path — no raw shell, but write/delete + nested-run reachable

- The stdio bridge is a stateless loopback HTTP proxy — it does not touch the filesystem or run shell itself.
- It does expose `write_file`/`delete_file`/`delete_project`/`start_run` to the outer agent, scoped to the project store. Review §3.3 before wiring; the destructive + nested-run tools are the reason this is not a pure read-only surface.

4.6 Verdict

SAFE TO ADOPT — containerized, rootless, localhost-only — with §5 hardening. NEEDS-MORE-REVIEW only for two opt-in modes: (a) the `docker-compose.linux.yml` host-networking + credential-mount override, and (b) `OD_CODEX_SANDBOX=danger-full-access`. BLOCKED: exposing the daemon beyond localhost without an authenticated proxy + token; building the image on the bare host (must go through §11.4.173).

---

5. Required hardening (apply all)

1. **Localhost only.** Keep the `127.0.0.1:PORT:7456` publish. Verify with `ss -ltnp | grep 7456` on every launch. Never bind a LAN/public IP without an authenticated reverse proxy AND `OD_API_TOKEN` (§4.1).
2. **Rootless podman, no sudo** (§11.4.161). Confirm `podman info` shows rootless; confirm cgroup `memory`/`pids` delegation so the limits bite (§2.0).
3. **Set `OD_API_TOKEN`** even on loopback (defense in depth; harmless). Never `OD_DISABLE_API_AUTH=1` outside a trusted authenticated proxy.
4. **Pin the image by digest** (`ghcr.io/nexu-io/od@sha256:<digest>`), not `:latest` (§2.2).
5. **Keep the base compose's guards**: `read_only: true`, `no-new-privileges:true`, `pids_limit`, `mem_limit` (bounded per §12.6), tmpfs `/tmp`, named volume for `/app/.od`. Do not relax them.
6. **Do NOT adopt `docker-compose.linux.yml`** (host networking + `~/.claude` credential mount + API-key env) unless an in-container agent runtime is genuinely required. If it is: scope credentials to a throwaway key, keep the mount read-only, keep `OD_BIND_HOST=127.0.0.1`, and treat it as a separate operator-gated decision (§11.4.122).

7. **Leave SSRF defaults**: empty `OD_ALLOWED_INTERNAL_HOSTS`; do not set `OD_CODEX_SANDBOX` (§4.4).
8. **No external egress beyond what a design run needs.** Consider a rootless podman network with restricted egress (containers-submodule `pkg/egress`/`pkg/network`) if the host policy requires it; the daemon's own reasoning-egress allowlist (§4.3) is the in-app second layer.
9. **MCP wiring stays loopback** (§3.1 `podman exec -i`, or §3.2 with `--daemon-url http://127.0.0.1:7456`). Never point the MCP bridge at a non-loopback daemon URL.
10. **Build only via §11.4.173** (containerized/distributed on the remote build host) or pull the pre-built image — never bare-host build.

---

6. DO NOT

> **DO NOT run the OpenDesign daemon directly on the bare host** (no `pnpm --filter @open-design/daemon start`, no host `od` daemon) — it is a network service that spawns coding-agent runtimes; run it only inside the rootless container (§11.4.173).
>
> **DO NOT expose the daemon beyond `127.0.0.1`.** No `0.0.0.0` publish, no LAN IP, no public port, no `network_mode: host` on a shared machine. Any remote reach MUST go through an authenticated reverse proxy / SSH tunnel / VPN **and** `OD_API_TOKEN` (vendor + §4.1).
>
> **DO NOT set `OD_DISABLE_API_AUTH=1`, `OD_CODEX_SANDBOX=danger-full-access`, or adopt `docker-compose.linux.yml`** without an explicit operator decision — each widens the blast radius (§4.4, §11.4.122).
>
> **DO NOT build the image on the bare host** — heavy native build, §11.4.173 requires the containerized/distributed build path; prefer the pinned pre-built image.

Sources verified 2026-07-09

Primary source of truth is the read-only clone at commit-state cloned into scratchpad (…/scratchpad/open-design/), files cited inline in §1. Cross-referenced against latest upstream (§11.4.99):

- https://github.com/nexu-io/open-design — repo overview (author-time daemon + MCP + 20+ CLIs via BYOK)
- https://github.com/nexu-io/open-design/blob/main/deploy/README.md — Docker deploy, localhost-bind guidance, "API unauthenticated for non-browser clients", image excludes agent CLIs
- https://github.com/nexu-io/open-design/blob/main/QUICKSTART.md — `od mcp install <agent>` one-liner + hosted `install.sh | sh -s <agent>`
- https://github.com/nexu-io/open-design/issues/3430 — per-agent MCP installer (tool surface: search files, get files, get artifacts, run plugins, list skills; `od mcp install <agent>`)
- https://deepwiki.com/nexu-io/open-design/12.5-docker-and-nix-deployment — Docker/Nix deployment, port 7456, env vars
- https://www.tutorialworks.com/podman-rootless-volumes/ — rootless podman named-volume storage under `$HOME`, ownership handling
- https://oneuptime.com/blog/post/2026-02-02-podman-security-configuration/view — rootless `--security-opt no-new-privileges:true` + `--read-only` + resource limits
- https://oneuptime.com/blog/post/2026-03-17-configure-volume-options-rootless-podman/view — `:U`/`:Z` volume flags, rootless permission handling
- https://oneuptime.com/blog/post/2026-01-27-podman-compose/view — podman-compose named volumes vs bind mounts

Negative finding: no upstream doc contradicts the localhost-only + rootless posture; the vendor's own guidance is stricter-or-equal to this recipe. No upstream rootless-podman-specific OpenDesign guide exists (vendor documents Docker + the Linux host-mount override); the rootless mapping in §2 is derived from the compose file + general rootless-podman references above and is marked where runtime-unverified.